

Teach Module 27: Transaction Management

Module Focus

Module 27 covers transaction management in Spring applications:

- Transactional integrity
- Declarative transactions with `@Transactional`
- Propagation behavior
- Rollback rules for checked and unchecked exceptions
- Isolation levels and concurrency trade-offs
- Consistency strategies in multi-step operations
- Using AI tools carefully to generate and validate transaction code

This README is written as teaching material in original wording and does not reuse the course material.

1. What Is A Transaction?

A transaction is a group of database operations treated as one unit.

Classic example: a bank transfer.

1. Subtract \$100 from Alice
2. Add \$100 to Bob

Both steps must succeed. If step 2 fails, step 1 must be undone. That "all succeed or all fail" behavior is the core idea.

Transactions are often explained with ACID:

- **Atomicity** : all steps succeed, or none do.
- **Consistency** : data remains valid before and after the transaction.
- **Isolation** : concurrent users do not corrupt each other's work.
- **Durability** : once committed, data survives system failure.

2. Spring's Main Tool: `@Transactional`

In Spring Boot, the most common way to manage transactions is declarative transaction management.

```
@Service
public class TransferService {

    private final AccountRepository accountRepository;

    public TransferService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findById(fromId)
            .orElseThrow();
```

```

        Account to = accountRepository.findById(toId)
            .orElseThrow();

        from.withdraw(amount);
        to.deposit(amount);

        accountRepository.save(from);
        accountRepository.save(to);
    }
}

```

With `@Transactional`, Spring opens a transaction before the method runs. If the method completes successfully, Spring commits. If an eligible exception is thrown, Spring rolls back.

Usually place `@Transactional` on service methods, not controllers. Controllers handle HTTP. Services handle business workflows.

3. Commit And Rollback

A commit means: make these database changes permanent.

A rollback means: undo everything done inside this transaction.

```

@Transactional
public void createOrder(OrderRequest request) {
    Order order = orderRepository.save(new Order(request.customerId()));

    inventoryService.reserveStock(request.items());

    paymentService.charge(request.paymentDetails());

    order.markPaid();
}

```

If payment fails, the system should not leave behind an order and inventory reservation as if everything worked.

Important: database transactions only roll back database work managed by the transaction. If you send an email or call an external payment API, the database rollback does not automatically undo that external side effect.

4. Rollback Rules

By default, Spring rolls back for:

```

RuntimeException
Error

```

Spring does not automatically roll back for checked exceptions unless configured.

```

@Transactional
public void process() throws IOException {
    repository.save(entity);
}

```

```
    throw new IOException("File failed");  
}
```

By default, this may still commit because `IOException` is a checked exception.

To force rollback:

```
@Transactional(rollbackFor = IOException.class)  
public void process() throws IOException {  
    repository.save(entity);  
    throw new IOException("File failed");  
}
```

Many business exceptions in Spring apps are unchecked:

```
public class InsufficientFundsException extends RuntimeException {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

Then rollback happens naturally.

5. Propagation

Propagation controls what happens when a transactional method calls another transactional method.

The default is:

```
@Transactional(propagation = Propagation.REQUIRED)
```

`REQUIRED` means: use the existing transaction if one exists; otherwise create a new one.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

`REQUIRES_NEW` means: pause the current transaction and start a separate one.

Example use case: audit logging.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)  
public void saveAuditLog(String message) {  
    auditRepository.save(new AuditLog(message));  
}
```

Even if the main business transaction rolls back, you may still want the audit record saved.

Use `REQUIRES_NEW` carefully because it commits independently.

6. Isolation Levels

Isolation controls how much one transaction can see changes from another transaction.

Common levels:

- `READ_UNCOMMITTED` : can see uncommitted changes. Rarely appropriate.
- `READ_COMMITTED` : only sees committed data. Common default in many databases.
- `REPEATABLE_READ` : if you read the same row twice, you get the same result.
- `SERIALIZABLE` : strongest isolation, behaves like transactions happen one at a time, but can be slower.

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void updateInventory(Long productId, int quantity) {
    Product product = productRepository.findById(productId)
        .orElseThrow();

    product.decreaseStock(quantity);
}
```

Higher isolation gives stronger correctness but may reduce performance.

7. Multi-Step Consistency

A common mistake is spreading one business workflow across many partial saves without thinking about failure.

Weak design:

```
public void checkout(CheckoutRequest request) {
    orderRepository.save(order);
    inventoryRepository.save(inventory);
    paymentRepository.save(payment);
}
```

If one step fails halfway, the system may be left in an inconsistent state.

Better:

```
@Transactional
public void checkout(CheckoutRequest request) {
    Order order = createOrder(request);
    reserveInventory(request);
    recordPayment(request);
    order.confirm();
}
```

The service method represents the full business action.

8. Common Spring Transaction Gotchas

Self-Invocation

This may not work as expected:

```

@Service
public class OrderService {

    public void outerMethod() {
        innerTransactionalMethod();
    }

    @Transactional
    public void innerTransactionalMethod() {
        // transaction may not start here
    }
}

```

Spring usually applies transactions through a proxy. A method calling another method inside the same class bypasses the proxy.

Better options:

- Put transactional boundaries on public service methods called from outside the bean.
- Move the transactional method into another service.

External Side Effects

This is risky:

```

@Transactional
public void registerUser(User user) {
    userRepository.save(user);
    emailService.sendWelcomeEmail(user.getEmail());
    throw new RuntimeException("Something failed");
}

```

The database save can roll back. The email cannot be unsent.

9. Practice Exercises

Exercise 1: Basic Money Transfer

Create an `Account` entity with `id`, `ownerName`, and `balance`.

Build:

```

@Transactional
public void transfer(Long fromAccountId, Long toAccountId, BigDecimal amount)

```

Practice goals:

- Load both accounts.
- Subtract from one account.
- Add to the other.
- Save both.
- Throw an exception if balance is insufficient.

- Confirm both balances roll back if something fails.

Exercise 2: Force A Rollback

Inside your transfer method, temporarily add:

```
throw new RuntimeException("Simulated failure");
```

After running the method, check the database.

Expected result: neither account balance should change.

Exercise 3: Checked Exception Rollback

Create:

```
@Transactional  
public void importPayments() throws IOException
```

Inside it:

1. Save a payment record.
2. Throw an `IOException`.

Observe whether the record is committed or rolled back.

Then change it to:

```
@Transactional(rollbackFor = IOException.class)
```

Expected lesson: Spring rolls back unchecked exceptions by default, but checked exceptions need explicit rollback configuration.

Exercise 4: Order Checkout Transaction

Create entities:

- `Order`
- `OrderItem`
- `InventoryItem`
- `Payment`

Build:

```
@Transactional  
public Order checkout(CheckoutRequest request)
```

Practice goals:

- Create an order.
- Reduce inventory.
- Record payment.
- Mark order as confirmed.

- Simulate failure after inventory update.
- Confirm the order and inventory updates roll back together.

Exercise 5: Audit Log With `REQUIRES_NEW`

Create an `AuditLog` entity.

In a separate service, write:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logEvent(String message)
```

Then call it from a main transaction that later fails.

Expected result:

- Main transaction rolls back.
- Audit log still commits.

Exercise 6: Compare `REQUIRED` vs `REQUIRES_NEW`

Create two methods:

```
@Transactional
public void parentOperation()
```

and:

```
@Transactional(propagation = Propagation.REQUIRED)
public void childOperation()
```

Then repeat with:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

Simulate failures in different places and observe which records remain in the database.

Exercise 7: Isolation Level Experiment

Create a `Product` entity with `stockQuantity`.

Open two transactions that try to reduce stock at the same time.

Try:

```
@Transactional(isolation = Isolation.READ_COMMITTED)
```

Then compare with:

```
@Transactional(isolation = Isolation.SERIALIZABLE)
```

Practice goal: observe how isolation affects concurrent updates.

Exercise 8: Self-Invocation Trap

Create this intentionally flawed service:

```
@Service
public class DemoService {

    public void outerMethod() {
        innerMethod();
    }

    @Transactional
    public void innerMethod() {
        // save something, then throw exception
    }
}
```

Call `outerMethod()` and observe whether the transaction behaves as expected.

Then fix it by moving `innerMethod()` into another service.

Exercise 9: Transactional Test

Write a test using `@SpringBootTest`.

Test that this method rolls back correctly:

```
assertThrows(RuntimeException.class, () -> service.transfer(...));
```

Then verify balances are unchanged.

Exercise 10: AI Review Exercise

Ask an AI tool to generate a transaction method for checkout or transfer.

Then review the code manually:

- Is `@Transactional` placed on the service layer?
- Are rollback rules correct?
- Are checked exceptions handled?
- Are external calls inside the transaction?
- Is propagation being used unnecessarily?
- Are there hidden partial commits?

10. Lab: Transaction Management In Spring Boot

Goal

Create a small Spring Boot app that proves transactions work by simulating success and failure cases.

Build a bank transfer feature where money moves from one account to another. If anything fails midway, both account balances must return to their original values.

You Will Practice

- `@Transactional`
- rollback on runtime exceptions
- rollback for checked exceptions
- transaction propagation with `REQUIRES_NEW`
- tests that prove rollback behavior

Suggested Project Setup

Create a Spring Boot project with:

- Spring Web
- Spring Data JPA
- H2 Database
- Lombok optional

Suggested packages:

```
com.example.transactions
com.example.transactions.account
com.example.transactions.audit
```

Step 1: Create Account Entity

```
@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String ownerName;

    private BigDecimal balance;

    protected Account() {
    }

    public Account(String ownerName, BigDecimal balance) {
        this.ownerName = ownerName;
        this.balance = balance;
    }

    public void withdraw(BigDecimal amount) {
        if (balance.compareTo(amount) < 0) {
            throw new IllegalStateException("Insufficient funds");
        }
        balance = balance.subtract(amount);
    }

    public void deposit(BigDecimal amount) {
```

```

        balance = balance.add(amount);
    }

    public Long getId() {
        return id;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public BigDecimal getBalance() {
        return balance;
    }
}

```

Step 2: Create Repository

```

public interface AccountRepository extends JpaRepository<Account, Long> {
}

```

Step 3: Create Transfer Service

```

@Service
public class TransferService {

    private final AccountRepository accountRepository;

    public TransferService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findById(fromId).orElseThrow();
        Account to = accountRepository.findById(toId).orElseThrow();

        from.withdraw(amount);
        to.deposit(amount);
    }
}

```

Inside a transaction, JPA dirty checking can detect changes to managed entities. Explicit `save()` calls are often unnecessary after loading entities in the same transaction.

Step 4: Seed Test Data

```

@Bean
CommandLineRunner seedAccounts(AccountRepository repository) {
}

```

```

return args -> {
    repository.save(new Account("Alice", new BigDecimal("1000.00")));
    repository.save(new Account("Bob", new BigDecimal("500.00")));
};
}

```

Step 5: Add A Failure Method

```

@Transactional
public void transferWithFailure(Long fromId, Long toId, BigDecimal amount) {
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();

    from.withdraw(amount);
    to.deposit(amount);

    throw new RuntimeException("Simulated failure after transfer");
}

```

Expected result: balances should not change.

Step 6: Write A Rollback Test

```

@SpringBootTest
class TransferServiceTest {

    @Autowired
    private TransferService transferService;

    @Autowired
    private AccountRepository accountRepository;

    @Test
    void transferWithFailureShouldRollbackBothAccounts() {
        Account alice = accountRepository.save(new Account("Alice", new
        BigDecimal("1000.00")));
        Account bob = accountRepository.save(new Account("Bob", new BigDecimal("500.00")));

        assertThrows(RuntimeException.class, () ->
            transferService.transferWithFailure(
                alice.getId(),
                bob.getId(),
                new BigDecimal("100.00")
            )
        );

        Account updatedAlice = accountRepository.findById(alice.getId()).orElseThrow();
        Account updatedBob = accountRepository.findById(bob.getId()).orElseThrow();

        assertEquals(new BigDecimal("1000.00"), updatedAlice.getBalance());
    }
}

```

```

        assertEquals(new BigDecimal("500.00"), updatedBob.getBalance());
    }
}

```

Step 7: Add Checked Exception Case

Create:

```

@Transactional
public void transferWithCheckedException(Long fromId, Long toId, BigDecimal amount) throws
IOException {
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();

    from.withdraw(amount);
    to.deposit(amount);

    throw new IOException("Checked exception failure");
}

```

Run a test. You may see that the transaction does not roll back.

Now fix it:

```

@Transactional(rollbackFor = IOException.class)
public void transferWithCheckedException(Long fromId, Long toId, BigDecimal amount) throws
IOException {
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();

    from.withdraw(amount);
    to.deposit(amount);

    throw new IOException("Checked exception failure");
}

```

Step 8: Add Audit Logging With New Transaction

Create an `AuditLog` entity:

```

@Entity
public class AuditLog {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String message;

    protected AuditLog() {
    }
}

```

```

    public AuditLog(String message) {
        this.message = message;
    }
}

```

Repository:

```

public interface AuditLogRepository extends JpaRepository<AuditLog, Long> {
}

```

Service:

```

@Service
public class AuditService {

    private final AuditLogRepository auditLogRepository;

    public AuditService(AuditLogRepository auditLogRepository) {
        this.auditLogRepository = auditLogRepository;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(String message) {
        auditLogRepository.save(new AuditLog(message));
    }
}

```

Then call it from the transfer service before throwing an exception.

Expected result:

- Account transfer rolls back.
- Audit log remains saved.

Lab Challenge

Add an endpoint:

```

POST /accounts/{fromId}/transfer/{toId}?amount=100

```

Then test:

1. successful transfer
2. insufficient funds
3. simulated failure
4. checked exception rollback
5. audit log surviving rollback

Completion Criteria

You are done when you can prove:

- successful transfers commit
- failed transfers roll back
- checked exceptions require `rollbackFor`
- `REQUIRES_NEW` commits independently
- tests verify the actual database state after failure

Checkpoint Quiz

1. Where should `@Transactional` usually go: controller, service, or repository?
2. Does Spring roll back by default for checked exceptions?
3. What does `Propagation.REQUIRED` mean?
4. Why can external API calls inside transactions be risky?
5. What is the trade-off of higher isolation levels?